PART 1 · MODULE B

Programming foundations: HTML, CSS, JavaScript, git and APIs

Every web profession starts with these languages and tools. You do not need to master them before you begin; you need to build and fix a small page.

B1. HTML: the structure

Simple explanation. HTML is the skeleton of a page. Headings, paragraphs, images, buttons and forms are HTML elements.

Everyday analogy. HTML is the layout of a printed menu: where the title goes, which items form a list, where the prices sit.

HTML

```
<!doctype html>
<html lang="en">
<head><meta charset="utf-8"><meta name="viewport" content="width=device-width, initial-scale=1">
<title>Asha's Bakery</title></head>
<body>
  <h1>Asha's Bakery</h1>
  <p>Fresh bread every morning.</p>
  <ul><li>Sourdough</li><li>Croissant</li></ul>
  <a href="mailto:hello@example.com">Order by email</a>
</body></html>
```

B2. CSS: the look

Simple explanation. CSS controls colour, size, spacing and layout. A rule selects elements and sets properties.

CSS

```
body { font-family: system-ui, sans-serif; margin: 0; padding: 16px; color: #1d1640; }
h1 { color: #6c4cf5; }
@media (min-width: 720px) { body { padding: 40px; max-width: 720px; margin: auto; } }
```

B3. Responsive design

A responsive page works on a phone (about 390 pixels wide) and a laptop (about 1280 pixels). Use the viewport meta tag, flexible widths (`max-width: 100%`) and media queries. Test both widths before delivery.

B4. JavaScript: behaviour

Simple explanation. JavaScript makes a page react: clicks, form checks and messages. It is a set of instructions the browser runs.

JAVASCRIPT

```
const form = document.querySelector('#enquiry');
const msg = document.querySelector('#message');
form.addEventListener('submit', (event) => {
  event.preventDefault();                       // stop the page reloading
  const digits = form.phone.value.replace(/\D/g, '');
  msg.textContent = digits.length < 10
```

```
    ? 'Please enter a 10-digit phone number.'
    : 'Thank you. We will call you soon.';
});
```

B5. Programming logic

Programs follow three ideas: **sequence** (steps in order), **decision** (if this, then that) and **repetition** (do this for each item). Write the logic in plain sentences before you write code.

B6. Git and GitHub

Simple explanation. Git records every change, so you can go back. GitHub stores a copy online so others can see and share it.

BASH

```
git init                      # start tracking this folder
git add index.html            # choose the files to save
git commit -m "Add bakery homepage"
git log --oneline             # see your history
```

B7. Command-line basics

The terminal runs commands instead of clicks. Useful ones: `pwd` (where am I), `ls` (list files), `cd folder` (move into a folder), `mkdir folder` (create a folder).

B8. Debugging

1. Read the error message exactly. It usually names the file and the line.
2. Reproduce the problem with the smallest example that shows it.
3. Use the browser developer tools (right-click, Inspect, Console).
4. Change one thing at a time, then test again.

B9. APIs

Simple explanation. An API is a messenger that lets two programs talk. You send a request in an agreed format and receive a structured reply, usually JSON.

JSON

```
{
  "name": "Sourdough",
  "price_inr": 180,
  "in_stock": true
}
```

B10. Basic databases

A table has columns (fields) and rows (records). A key identifies each row. SQL is the language for reading and writing tables.

SQL

```
CREATE TABLE products (id INTEGER PRIMARY KEY, name TEXT, price_inr INTEGER);
INSERT INTO products VALUES (1, 'Sourdough', 180);
SELECT name, price_inr FROM products WHERE price_inr < 250;
```

OFFICIAL LEARNING LINKS (NOT CHECKED LIVE IN THIS SESSION)

- MDN Web Docs: Learn web development
- Pro Git (free book)
- GitHub Skills

ASSIGNMENT

Build and publish a one-page bakery site

Business problem: A bakery needs a simple online menu and an order form that checks the phone number before it sends.

Steps

1. Create `index.html` with a heading, a three-item menu list and a form with name, phone and message fields.
2. Create `style.css` so the page is readable on a phone and a laptop.
3. Create `script.js` that stops the form reloading and checks for at least 10 digits in the phone field.
4. Save the work with git and make at least two commits.

Expected output: A folder with three files and a git history of at least two commits. A short phone number shows an error; a valid one shows the thank-you message.

SOLUTION (ANSWER KEY)

HTML

```
<!-- index.html -->
<!doctype html><html lang="en"><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Asha's Bakery</title><link rel="stylesheet" href="style.css"></head>
<body>
<h1>Asha's Bakery</h1>
<ul><li>Sourdough - Rs 180</li><li>Croissant - Rs 90</li><li>Rye loaf - Rs 220</li></ul>
<form id="enquiry">
  <label>Name <input name="name" required></label>
  <label>Phone <input name="phone" required></label>
  <label>Message <textarea name="message"></textarea></label>
  <button>Order</button>
</form>
<p id="message" role="status"></p>
<script src="script.js"></script>
</body></html>
```

CSS

```
*, *::before, *::after { box-sizing: border-box; }
body { font-family: system-ui, sans-serif; margin: 16px; color: #1d1640; }
h1 { color: #6c4cf5; }
label { display: block; margin: 8px 0; }
input, textarea { width: 100%; max-width: 360px; padding: 8px; }
@media (min-width: 720px) { body { max-width: 720px; margin: 40px auto; } }
```

JAVASCRIPT

```
// script.js
const form = document.querySelector('#enquiry');
const msg = document.querySelector('#message');
form.addEventListener('submit', (e) => {
  e.preventDefault();
  const digits = form.phone.value.replace(/\D/g, '');
  msg.textContent = digits.length < 10
    ? 'Please enter a 10-digit phone number.'
    : 'Thank you. We will call you soon.';
});
```

Why it works: HTML gives the structure, CSS adapts the layout to the screen width, and the script stops the browser reloading the form. The digit check removes spaces and symbols first, so '+91 98765-43210' counts as 12 digits and passes.

Part of **Master Freelancing: Development and IT**. Back to the chapter list on skillnavora.in · Open the full textbook. Official links in this chapter were not checked live; confirm before relying on them.